DE 424 Mini Project - Find the Wumpus

A Probabilistic Graphical Model for Wumpus Tracking

Kehan Taljaard 26867206

14 May 2026

Declaration

I, Kehan Taljaard, declare that this report and the work presented in it are my own work, except where explicitly acknowledged below. I have not received or accepted information from any source not properly acknowledged in this declaration or elsewhere in the report.

Use of AI assistance. [FILL IN: specific account of AI tool use.]

Other sources. The DE 424 lecture notes (Weeks 1–10), the *emdw* user guide and the example code provided with the project, and the textbook by Barber [1] (in particular Chapter 23 on dynamic models) were consulted for theoretical background and library API usage.

ECSA Graduate Attributes

GA 1 – Problem solving. The wumpus tracking problem is formulated from first principles as a hidden Markov model with a uniform-random-walk transition and conditionally independent per-cell Bernoulli observations (Section 2). The model structure is motivated by the problem's Markov dynamics and the conditional independence of the detection grid given the unobserved position; references to the literature underpinning this formulation are given throughout [1, 2]. A key simplification , collapsing $(T+1)G$ binary detection RVs into $T+1$ unary evidence factors on the unobserved variables, is derived from first principles in Section 2.5. Design trade-offs (exact inference on a chain, point-estimate parameter learning by EM rather than full Bayesian inference, scoping to datasets 1–3) are analysed in Sections 2.8 and 4.

GA 2 – Application of scientific and engineering knowledge. The solution applies probability theory (joint factorisation, marginalisation, conditional independence), graphical-model theory (Bayes networks, factor graphs, cluster graphs), and inference algorithms (sum-product belief propagation, the Expectation–Maximisation algorithm) to the engineering problem of tracking from inaccurate measurements. The C++ implementation using the `emdw` library (Section 3) operationalises the design. Convergence behaviour and numerical considerations are reported in Section 4.

1 Problem description

A wumpus performs a random walk on a $W \times H$ grid over $T+1$ discrete time steps. Its position is unobserved. At each time step, every cell of the grid emits a binary detection: the wumpus's cell emits a 1 with probability p_w and a 0 otherwise, while every other cell emits a 1 with probability p_c and a 0 otherwise. The entire detection grid is observed at every time step. The task is to infer the wumpus's trajectory from the sequence of detection grids.

Five datasets are provided. This work addresses datasets 1, 2 and 3. The scoping decision is motivated in Section 2.8.

2 Solution design

2.1 Notation

Symbol	Meaning
W, H	grid width and height (columns and rows)
$G = W \cdot H$	total number of cells
T	final time step index (T+1 steps in total)
$\mathcal{X}$	state space, $\{0, 1, \ldots, G-1\}$
$X_t \in \mathcal{X}$	unobserved wumpus cell index at time t
$D_t^{(i)} \in \{0, 1\}$	detection RV at cell i at time t
$d_t^{(i)} \in \{0, 1\}$	observed value of $D_t^{(i)}$
$\mathbf{d}_t$	full detection grid at time t
p_w, p_c	detection probabilities (wumpus cell / other cells)
$\gamma_t(x)$	posterior $P(X_t = x \mid \mathbf{d}_{0:T})$

2.2 Random variables and Bayes network structure

The unobserved variables are the wumpus positions $X_0, \ldots, X_T$, each a discrete RV over the G cells under a position encoding $i = \text{row} \cdot W + \text{col}$. The observed variables are the detections $D_t^{(i)}$ for each cell i and time t. The conditional independence assumptions encoded by the model are:

- **Markov dynamics**: $X_{t+1} \perp\!\!\!\perp X_{0:t-1} \mid X_t$.
- **Conditionally independent detections**: given X_t, the per-cell detections at time t are mutually independent of each other and of all detections at other times.

The resulting Bayes net is shown in Figure 1. The joint distribution factorises as

$$p(X_{0:T}, \{D_t^{(i)}\}) = p(X_0) \prod_{t=0}^{T-1} p(X_{t+1} \mid X_t) \prod_{t=0}^{T} \prod_{i=0}^{G-1} p(D_t^{(i)} \mid X_t). \tag{1}$$

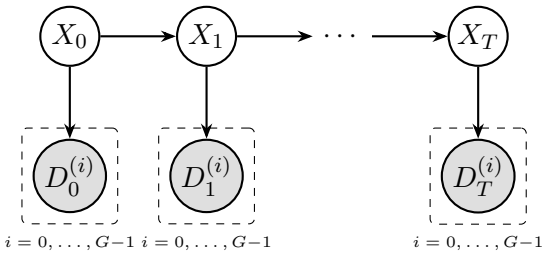

Figure 1: Bayes network for the wumpus tracking model. Each unobserved X_t has G binary detection children.

2.3 Initial-state prior

In the absence of information about the wumpus's starting cell, a uniform prior is used:

$$p(X_0 = x) = \frac{1}{G}, \qquad \forall x \in \mathcal{X}. \tag{2}$$

2.4 Transition factor

At each step the wumpus selects a direction $d \in \{U, D, L, R\}$ uniformly and attempts a move. If the target cell is out of grid bounds the move fails and the wumpus remains in place. Marginalising out the unobserved direction, the transition factor is given by:

$$p(X_{t+1} = j \,|\, X_t = i) \;=\; \tfrac{1}{4} \sum_{d \in \{U,D,L,R\}} \mathbb{I}[\,\mathrm{move}(i, d) = j\,]$$

where $\mathrm{move}(i, d)$ returns the target cell of move d from i, clipped back to i if the target is out of bounds.
The factor is sparse: each row has at most five non-zero entries (at most four in-bounds neighbours plus the cell's own mass carried back from clipped moves). The factor is time-homogeneous, so a single sparse table is built at the beginning and used for every transition.

2.5 Observation factor collapse

Because every detection $D_t^{(i)}$ is observed, the $(T{+}1)G$ emission factors can be folded into one unary factor per time step on X_t, removing the detection RVs from the graph entirely.

By the conditional independence of detections given the wumpus position (Section 2.2), the joint emission likelihood at time t factorises as:

$$p(\mathbf{d}_t \,|\, X_t = x^\star) = \prod_{i=0}^{G-1} p\big(D_t^{(i)} = d_t^{(i)} \,|\, X_t = x^\star\big),$$

and each per-cell likelihood depends only on whether cell i contains the wumpus:

$$p\big(D_t^{(i)} = d_t^{(i)} \,|\, X_t = x^\star\big) = \begin{cases} p_w^{d_t^{(i)}} (1{-}p_w)^{1-d_t^{(i)}} & i = x^\star, \\ p_c^{d_t^{(i)}} (1{-}p_c)^{1-d_t^{(i)}} & i \neq x^\star. \end{cases}$$

Next, we can factor out the $i \neq x^\star$ terms, which do not depend on $x^\star$ and so:

$$p(\mathbf{d}_t \,|\, X_t = x^\star) = p_w^{d_t^{(x^\star)}} (1{-}p_w)^{1-d_t^{(x^\star)}} \cdot \overbrace{\prod_{i \neq x^\star} p_c^{d_t^{(i)}} (1{-}p_c)^{1-d_t^{(i)}}}^{①}$$

Where ① is expanded to the following to enable removal of terms independent of $x^\star$:

$$\frac{\prod_{i=0}^{G-1} p_c^{d_t^{(i)}} (1{-}p_c)^{1-d_t^{(i)}}}{p_c^{d_t^{(x^\star)}} (1{-}p_c)^{1-d_t^{(x^\star)}}}$$

Now:

$$p(\mathbf{d}_t \,|\, X_t = x^\star) = \underbrace{\Big[\prod_{i=0}^{G-1} p_c^{d_t^{(i)}} (1{-}p_c)^{1-d_t^{(i)}}\Big]}_{C_t,\ \text{independent of } x^\star} \cdot \frac{p_w^{d_t^{(x^\star)}} (1{-}p_w)^{1-d_t^{(x^\star)}}}{p_c^{d_t^{(x^\star)}} (1{-}p_c)^{1-d_t^{(x^\star)}}}.$$

C_t does not depend on $x^\star$ and cancels in any posterior over X_t. We define the unary observation factor:

$$\phi_t(X_t = x^\star) \;\propto\; \begin{cases} p_w / p_c & d_t^{(x^\star)} = 1, \\ (1 - p_w)/(1 - p_c) & d_t^{(x^\star)} = 0. \end{cases}$$

This is a length-G table per time step with two distinct values. The $G(T{+}1)$ binary detection RVs are absent from the graph used for inference, which becomes a chain of $T{+}1$ unobserved variables with one unary evidence factor per step — a factor of G smaller than the naive representation. The brief explicitly recommends exploiting always-observed RVs in this way.

2.6 Cluster graph and inference algorithm

The factor list (one prior, T transition factors, $T{+}1$ collapsed observation factors) is handed to `emdw`'s `ClusterGraph` constructor with either the `LTRIP` or the `JTREE` construction tag. For the chain-structured factor graph induced by this model, the resulting cluster graph is a chain of pair clusters $\{X_t, X_{t+1}\}$ joined by singleton sepsets $\{X_{t+1}\}$, so sum-product belief propagation is exact and converges after one forward and one backward sweep. `loopyBP_CG` on this tree-structured CG behaves as standard forward–backward.

Sum-product marginal inference was chosen over max-product MAP for two reasons: (i) the per-time-step smoothed marginal $\gamma_t(x) = p(X_t \mid \mathbf{d}_{0:T})$ is the natural output for visualisation and for the classifier-style quantitative evaluation suggested by the brief; (ii) the smoothed marginals are exactly the quantity required by the E-step of the EM algorithm used for dataset 3 (Section 2.7).

2.7 Parameter learning for unknown p_w, p_c (dataset 3)

For dataset 3 the detection parameters are unknown. We treat them as **point estimates** learned by the Expectation–Maximisation algorithm. This choice keeps the graph chain-structured (so inference remains exact in each E-step) and reuses the same forward–backward machinery. The alternative of placing Beta priors on p_w, p_c and performing full Bayesian inference would have required either discretisation or conjugate updates and was judged unnecessary for the project's scope; treating the parameters as constants (hand-tuned) was rejected because the problem explicitly states that the parameters are unknown.

The complete-data log-likelihood (dropping terms independent of $\theta = (p_w, p_c)$) reduces to a sum of Bernoulli log-likelihoods at the wumpus cell and at non-wumpus cells. Replacing the unknown trajectory by its posterior yields the expected complete-data log-likelihood

$$Q(\theta \mid \theta^{(k)}) = N_w^{(1)} \log p_w + N_w^{(0)} \log(1{-}p_w) + N_c^{(1)} \log p_c + N_c^{(0)} \log(1{-}p_c) + \text{const},$$

where the soft sufficient statistics are $A_t = \sum_x \gamma_t^{(k)}(x)\, d_t^{(x)}$ (expected detection at the wumpus cell at time t), $N_w^{(1)} = \sum_t A_t$, $N_w^{(0)} = (T{+}1) - N_w^{(1)}$, $N_c^{(1)} = \sum_t (D_t - A_t)$ with $D_t = \sum_i d_t^{(i)}$, and $N_c^{(0)} = (T{+}1)(G{-}1) - N_c^{(1)}$. Setting $\partial Q/\partial p_w = \partial Q/\partial p_c = 0$ gives:

M-step

$$p_w^{(k+1)} = \frac{\sum_t A_t}{T+1}, \qquad p_c^{(k+1)} = \frac{\sum_t (D_t - A_t)}{(T+1)(G-1)}.$$

The E-step rebuilds the observation factors ϕ_t under the current parameters, runs BP on the chain, and reads off the smoothed marginals $\gamma_t^{(k)}$. The algorithm alternates between E and M until $\max\big(|p_w^{(k+1)} - p_w^{(k)}|, |p_c^{(k+1)} - p_c^{(k)}|\big) < 10^{-4}$ or 50 iterations are reached. Parameter estimates are clamped to $[10^{-6}, 1{-}10^{-6}]$ to avoid singular observation factors at boundary values. EM is initialised at $(p_w^{(0)}, p_c^{(0)}) = (0.9, 0.1)$; any initialisation with $p_w > p_c$ produces informative observation factors that bootstrap the alternation toward the correct likelihood mode.

2.8 Scope of solution

Following the brief's first tip — "don't be too ambitious" — this work addresses datasets 1–3 in depth and does not extend to datasets 4–5, which additionally feature unknown occupied cells. Two extensions to the model were considered for the occupied-cell case: (i) joint inference over the unobserved trajectory and binary map RVs $M_c \in \{0, 1\}$, which would render the graph loopy via M_c appearing in multiple transition factors and so require approximate inference; and (ii) alternating EM-style estimation treating the map as a parameter, which preserves the tree-structured X chain but requires a non-trivial new M-step derivation for the map probabilities. Both introduce failure

modes for which insufficient time remained in the project window; choosing depth over breadth permitted thorough analysis of datasets 1–3 (Section 4), including `LTRIP` vs `JTREE` comparison and verification of forward–backward smoothing behaviour at trajectory endpoints.

3 Implementation

The implementation comprises two main programs and a shared helper library:

- `miniproject.cc` — datasets 1–2 (known parameters)
- `miniproject_q3.cc` — dataset 3 (EM-based parameter learning)
- `miniproject_helpers.cc` — I/O for detection grids, ground truth, marginals, and trajectories

Both main programs share the same skeleton — load detections, build factors, construct cluster graph, run BP, extract per-time-step marginals, write results to disk — with the EM variant wrapping factor build, CG construction, BP, and marginal extraction in an outer loop implementing the M-step of Section 2.7. Each program takes the dataset directory, output directory, initial (p_w, p_c), and an optional cluster-graph type (`LTRIP`, `JTREE` or `BETHE`) on the command line.

3.1 Variable identifiers and shared cell domain

Each X_t is assigned a sequential integer identifier, and all X_t share a single length-G cell domain instantiated once:

```
vector<RVIdType> X(lastT + 1);
for (int t = 0; t ⩽ lastT; ++t) X[t] = static_cast<RVIdType>(t);

rcptr<vector<T>> cellDom(new vector<T>(G));
for (int i = 0; i < G; ++i) (*cellDom)[i] = i;
```

3.2 Building the factors

Prior. The uniform prior on X_0 is encoded compactly by giving an empty sparse map with a default-entry probability of $1/G$, so every entry of the length-G table is automatically $1/G$:

```
map<vector<T>, FProb> emptyProbs;
rcptr<Factor> prior = uniqptr<DT>(new DT(
    {X[0]}, {cellDom}, 1.0 / static_cast<double>(G), emptyProbs));
```

Transition factor. The transition table is built by enumerating each source cell and accumulating $\frac{1}{4}$ of mass per direction into the appropriate destination. Out-of-bounds moves are mapped back to the source cell by `clipMove`, which automatically generates the corner/edge self-loop mass without an explicit case split:

```
auto clipMove = [W, H, &cellOf](int row, int col, int drow, int dcol) {
  int nr = row + drow, nc = col + dcol;
  if (nr < 0 || nr ⩾ H) nr = row;
  if (nc < 0 || nc ⩾ W) nc = col;
  return cellOf(nr, nc);
};

map<vector<T>, FProb> transProbs;
for (int i = 0; i < G; ++i) {
  const int r = rowOf(i), c = colOf(i);
  for (int d = 0; d < 4; ++d) {
    const int j = clipMove(r, c, directions[d][0], directions[d][1]);
    transProbs[{T(i), T(j)}].prob += 0.25;
```

```
  }
}
```

The sparse map is built once and reused across all T transition factor instantiations, exploiting the time-homogeneity of the dynamics.

Observation factor. Each ϕ_t takes one of the two values $\phi_{\text{on}} = p_w/p_c$ or $\phi_{\text{off}} = (1-p_w)/(1-p_c)$ per cell, determined by the detection grid at time t:

```
const double phiOn  = cfg.pw / cfg.pc;
const double phiOff = (1.0 - cfg.pw) / (1.0 - cfg.pc);

for (int t = 0; t <= lastT; ++t) {
  map<vector<T>, FProb> phiProbs;
  for (int row = 0; row < H; ++row)
    for (int col = 0; col < W; ++col) {
      const int j = cellOf(row, col);
      phiProbs[{T(j)}] =
          (detections[t][row][col] == 1) ? phiOn : phiOff;
    }
  rcptr<Factor> phi = uniqptr<DT>(new DT(
      {X[t]}, {cellDom}, defProb, phiProbs));
  factors.push_back(phi);
}
```

3.3 Cluster graph and BP

The cluster-graph construction algorithm is selectable at runtime, enabling the `LTRIP`/`JTREE` comparison reported in Section 4:

```
map<RVIdType, AnyType> obsv;  // evidence baked into phi_t; no RV-level obs
if (cfg.cgType == "JTREE")
  cgPtr = new ClusterGraph(ClusterGraph::JTREE, factors, obsv);
else if (cfg.cgType == "BETHE")
  cgPtr = new ClusterGraph(ClusterGraph::BETHE, factors, obsv);
else
  cgPtr = new ClusterGraph(ClusterGraph::LTRIP, factors, obsv);

map<Idx2, rcptr<Factor>> msgs;  MessageQueue msgQ;
unsigned nMsgs = loopyBP_CG(*cgPtr, msgs, msgQ);
```

3.4 Extracting marginals and the EM loop

For each t the smoothed marginal γ_t is obtained by querying the cluster graph for the posterior over X_t and reducing to each cell value in turn:

```
rcptr<Factor> q = queryLBP_CG(cg, msgs, {X[t]})->normalize();
for (int j = 0; j < G; ++j) {
  rcptr<Factor> reduced = q->observeAndReduce({X[t]}, {T(j)});
  gammas[t][j] = reduced->potentialAt(
      emdw::RVIds{}, emdw::RVVals{}, true);
}
```

In `miniproject_q3.cc` the same E-step logic is wrapped in an outer EM loop. After each E-step the soft sufficient statistics A_t are computed directly from `gammas[t][j]` and the detection grid, and the M-step updates (p_w, p_c) per Equation (2.7):

```
double sumA = 0.0, sumDminusA = 0.0;
for (int t = 0; t ⩽ lastT; ++t) {
  double At = 0.0;
  for (int row = 0; row < H; ++row)
    for (int col = 0; col < W; ++col)
      At += gammas[t][cellOf(row, col)]
            * static_cast<double>(detections[t][row][col]);
  sumA       += At;
  sumDminusA += static_cast<double>(Dt[t]) - At;
}
double newPw = sumA / static_cast<double>(lastT + 1);
double newPc = sumDminusA
             / (static_cast<double>(lastT + 1)
              * static_cast<double>(G - 1));
newPw = clamp(newPw, PARAM_EPS, 1.0 - PARAM_EPS);
newPc = clamp(newPc, PARAM_EPS, 1.0 - PARAM_EPS);
```

3.5 Output

For each time step the smoothed marginal is reshaped from its length-G form into a $W \times H$ matrix of probabilities and written to `marginal_<tt>.txt`; the per-step argmax cell forms a trajectory file in the ground-truth coordinate format $(x, y) = (\text{col}, \text{row})$ defined by the brief.

4 Results and analysis

4.1 Evaluation methodology

The inferred trajectory is evaluated against the provided ground truth using:

- **Per-step accuracy** — fraction of time steps for which $\arg\max_x \gamma_t(x)$ equals the ground-truth cell.
- **Mean ℓ_2 cell distance** — average Euclidean distance (in cells) between the inferred argmax cell and the ground-truth cell across time steps.
- **Top-k accuracy** — fraction of time steps for which the ground-truth cell lies in the k most probable cells under γ_t, reported for $k = 1, 3, 5$.

Qualitative evaluation uses posterior heatmaps for representative time steps with the ground-truth cell overlaid.

4.2 Dataset 1 (5×5, p_w=0.95, p_c=0.05)

[FILL IN: 2–3 sentences summarising the result, referencing Figure 2 and the corresponding row of Table 1.]

Figure 2: Posterior heatmaps for dataset 1 at representative time steps; ground-truth cell marked with a black square. [REPLACE WITH FIGURE]

4.3 Dataset 2 (20×20, p_w=0.9, p_c=0.1)

[FILL IN: 2–3 sentences for dataset 2. Note any observations about the larger grid or the smoothing behaviour at trajectory endpoints if relevant.]

4.4 Dataset 3 (10×20, EM-learned parameters)

[FILL IN: 2–3 sentences for dataset 3. Include the converged values of (p_w, p_c) and the number of EM iterations.]

Dataset	Per-step accuracy	Mean cell distance	Top-3 acc.	Top-5 acc.
1	[XX.X%]	[X.XX]	[XX.X%]	[XX.X%]
2	[XX.X%]	[X.XX]	[XX.X%]	[XX.X%]
3	[XX.X%]	[X.XX]	[XX.X%]	[XX.X%]

Table 1: Quantitative results across the three datasets addressed.

Figure 3: Convergence of p_w and p_c under EM on dataset 3. [REPLACE WITH FIGURE]

EM converged after [N] iterations to $(\hat{p}_w, \hat{p}_c) = ([X.XX], [X.XX])$. The parameter deltas decreased monotonically across iterations, consistent with the theoretical guarantee that EM is monotone in the marginal data log-likelihood — a useful sanity check given the non-trivial E-step.

4.5 Smoothing behaviour and cluster-graph choice

[FILL IN if applicable: report the `LTRIP` vs `JTREE` comparison, including any observations about the per-time-step posterior sharpness at the start, middle, and end of the trajectory. The brief flags "testing of additional aspects (e.g. convergence behaviour)" and "design, implementation and comparison of several different solution variations" as contributing to a higher mark.]

5 Conclusion

[FILL IN: brief summary paragraph. Suggested points to include: scope (datasets 1–3); key design decisions (HMM Bayes net structure, factor-graph collapse of detection RVs into per-step unary factors, point-estimate EM for unknown parameters, exact sum-product BP on a chain cluster graph); brief acknowledgement of the un-tackled extensions to occupied cells in datasets 4–5 with an indication of the natural extension path (loopy BP over binary map RVs vs alternating EM treating the map as a parameter).]

References

[1] D. Barber, *Bayesian Reasoning and Machine Learning*. Cambridge University Press, 2012.

[2] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Prentice Hall, 2022.

[3] H. Koen, H. Roodt, and A. De Waal, "An expert-driven causal model of the rhino poaching problem," *Ecological Modelling*, vol. 347, pp. 29–39, 2017.